Phase 4 — Concept Deep-Dives

Two concepts underpin the notification layer, each covered with mechanics, worked examples, and senior-level interview Q&A:

10. Pub/Sub fan-out (exchanges, queues, and consumer groups)
11. Event-carried state transfer (and per-consumer idempotency strength)

10. Pub/Sub Fan-Out

The principle

Fan-out means one published event is delivered to *multiple independent consumers*, each processing it for its own purpose. In RabbitMQ the mechanism is the exchange/queue split: publishers send to an **exchange** with a routing key; the exchange copies the message into every **queue** whose binding matches. The publisher neither knows nor cares how many queues are bound. In this platform, `payment.succeeded` was already consumed by the Order service (to advance the saga); Phase 4 binds a second queue — `notification.events` — to the same exchange and the same routing keys, and the Notification service starts receiving copies of the same events. Not one line of producer code changed.

The minute sub-concepts

The exchange/queue split is the whole design. A queue delivers each message to exactly ONE of its consumers. An exchange copies each message to EVERY bound queue. These compose into the two fundamental patterns:

- one queue, N consumers -> **competing consumers** (work distribution: each message processed once, by whichever replica takes it)
- N queues, one exchange -> **fan-out** (broadcast: each message processed once *per queue*)

A "consumer group" is therefore just "a queue": every service that needs its own copy of the stream declares its own queue; every replica of that service shares that queue.

Scaling interacts with both at once. Scale notification-service to 3 replicas and all 3 consume `notification.events` — competing consumers within the group, so each event still produces exactly one notification. The saga services are unaffected: their queues are separate. Fan-out across groups, competition within a group.

Topic exchange binding semantics. Bindings are patterns over dot-separated routing keys: `*` matches exactly one word, `#` matches zero or more. `order.*` matches `order.created` but not `order.item.added`; `order.#` matches both. This project binds explicit keys (`order.created`, `payment.succeeded`, `payment.failed`, `inventory.rejected`) rather than `#` — subscribing narrowly is deliberate: a consumer that takes everything becomes coupled to every future event's existence.

Producers must not know about consumers — verify it. The test of true decoupling is exactly what Phase 4 did: add a consumer with zero producer changes. If adding a consumer requires touching the producer, you have point-to-point messaging wearing a pub/sub costume.

Each queue has independent depth, lag, and failure. One slow consumer group backs up ITS queue only; the saga queues keep draining. This isolation is fan-out's operational superpower — and its trap: an abandoned bound queue (consumer removed, binding left behind) accumulates messages forever. Monitor per-queue depth (Phase 5).

Durability is per-layer. Durable exchange + durable queue + persistent messages are three separate flags; a persistent message in a non-durable queue still dies with the broker. This project sets all three for every saga-related entity.

Delivery to N queues is not a transaction. The broker copies the message to each bound queue individually; consumers see it at different times. Fan-out consumers must not assume any ordering *between* groups — only per-queue FIFO-ish ordering within one.

Practical worked example

The entire Phase 4 "integration" is one `subscribe` call in the Notification service:

```
await subscribe(
  'notification.events',                        // OUR queue (consumer group)
  ['order.created', 'payment.succeeded',
   'payment.failed', 'inventory.rejected'],     // bindings on the SAME exchange
  onEvent,
);
```

Compare with the Order service, which was already subscribed to some of the same keys:

```
await subscribe('order.payment-succeeded', ['payment.succeeded'], ...);
```

Same exchange (`ecommerce.events`), same routing key (`payment.succeeded`), two different queues -> RabbitMQ copies each `payment.succeeded` into both. Prove it live:

```
docker compose ... exec rabbitmq rabbitmqctl list_bindings source_name destination_name routing_key
```

You'll see `ecommerce.events -> order.payment-succeeded (payment.succeeded)` AND `ecommerce.events -> notification.events (payment.succeeded)`. Place a `BOOK-001` order and both consumers log their own handling of the same event: the Order service flips the order to CONFIRMED; the Notification service "sends" *Your order is confirmed*.

10 senior interview questions

1. **A message must be processed by every service that cares, but only once

per service, even when services are scaled. Design the topology.** (One exchange; one durable queue per service/consumer-group bound to relevant keys; replicas of a service share its queue — fan-out across queues, competing consumers within one.)

2. **What's the difference between a RabbitMQ fanout exchange and achieving

fan-out with a topic exchange?** (A fanout exchange ignores routing keys and copies to all bound queues; a topic exchange fans out only to queues whose patterns match — selective fan-out. This project uses topic so each group subscribes narrowly; the DLX here is a true fanout exchange.)

3. **How does RabbitMQ's model compare to Kafka's for fan-out?** (Kafka:

consumers pull from a shared log; each consumer group tracks its own offset; fan-out is free and replayable. RabbitMQ: broker pushes copies into per-group queues; no replay after ack. Kafka suits event streaming/ reprocessing; RabbitMQ suits task/command distribution and complex routing.)

4. **What happens if a bound queue has no consumer for a week?** (It

accumulates all matching messages — unbounded growth, broker memory/disk pressure. Mitigate: monitor queue depth, set max-length or TTL policies, and delete bindings when retiring consumers.)

5. **New consumer group added today — can it see yesterday's events?** (No.

Queues only receive messages published after the binding exists. If replay matters, you need an event store/log — Kafka, or an outbox table you can re-drain — and this trade-off should be named at design time.)

6. **Does adding consumer groups slow producers down?** (Publishing cost

grows slightly with copies, but producers don't wait for consumers; confirm mode waits only for broker persistence. The real risk is broker resource pressure from many/slow queues, not producer latency.)

7. **How do you prevent a 'catch-all' consumer becoming a coupling point?**

(Bind explicit keys rather than `#`; treat event schemas as published contracts; version events additively — new fields, not changed meanings — so old consumers keep working.)

8. **Where does ordering break in fan-out systems?** (Order is per-queue at

best; redelivery and multiple replicas reorder within a group; between groups there is no ordering at all. Consumers must rely on state machines/versions, not arrival order — the saga's persisted status does exactly this.)

9. **One consumer group needs the event 99.999% reliably; another is

best-effort. Same exchange?** (Yes — reliability is per-queue: the critical group gets durable queue + persistent messages + DLX + monitoring; the best-effort group can use TTL/max-length. Fan-out lets consumers choose their own guarantees.)

10. **How would you test that your system is genuinely pub/sub decoupled?**

(Add a new consumer group in a sprint with zero producer diffs — Phase 4 is literally this test. Also: kill a consumer group and verify producers and sibling groups are unaffected.)

11. Event-Carried State Transfer

The principle

Consumers of events often need more context than the triggering event carries. `payment.failed` says *which order* failed — but a notification needs *which user* to tell. There are three ways to get it: call the owning service back (synchronous coupling returns through the back door), share the database (worst option — breaks ownership), or **carry the state in the events themselves** and let each consumer accumulate the slice it needs. Phase 4 does the third: `order.created` now carries `userId`, and the Notification service remembers `orderId -> user` locally, so when `payment.failed` arrives later it can attribute the notification without asking anyone.

The minute sub-concepts

Thin vs fat events. A thin event is a notification-with-an-ID ("order 1234 changed — come ask me"); a fat event carries the state ("order 1234, user 42, items [...] was created"). Thin keeps payloads small but forces consumers to call back (re-coupling, and a read-your-own-write race: the callback may hit a replica that hasn't seen the write). Fat decouples fully but bloats the bus and can smear ownership. The engineering answer is *deliberately sized* events: carry what downstream consumers legitimately need — here, `userId` — not the whole aggregate.

Additive schema evolution. Adding `userId` to `order.created` broke nothing: the Inventory service destructures `{ orderId, items }` and ignores extra fields. That's the contract discipline for event payloads: add fields freely; never rename, remove, or change the meaning of existing ones without a versioning strategy (new event type or explicit version field).

Consumer-local state is a cache built from the stream. The Notification service's `orderId -> user` map is not a second source of truth — it's a projection, rebuildable (in principle) by replaying events, and safe to lose in proportion to what it powers. This is the same idea that scales up to CQRS read models; here it's a bounded in-memory map because notifications tolerate that.

Bound your projections. In-memory maps grow forever unless capped. This service caps at 1000 entries with insertion-order eviction (a Map iterates in insertion order — a poor man's LRU). Naming the bound, and what happens past it (a very old order's late event would notify `user:unknown`), is the senior move; unbounded caches are memory leaks with better PR.

Idempotency strength is a per-consumer decision. At-least-once delivery threatens every consumer with duplicates, but the correct defense differs by stakes. Inventory mutates money-adjacent state -> durable `processed_ reservations` table inside the same DB transaction. Notification sends an email -> a bounded in-memory `(event, orderId)` dedupe set suffices; the worst post-restart failure is one repeated email. Matching the mechanism to the blast radius — rather than maximal machinery everywhere — is the skill interviewers probe.

Loss semantics of a stateless consumer. Restart the Notification service and both its maps vanish. Consequences: a duplicate email is possible (dedupe set lost), and events for pre-restart orders attribute to `user:unknown` (context map lost). Both are acceptable *for this consumer* and would be unacceptable for inventory. Same events, different consumers, different durability — by design, and the design is defensible out loud.

Practical worked example

The producer side is one enriched payload in the Order service's outbox insert:

```
JSON.stringify({ orderId, userId: input.userId, items: input.items })
```

The consumer side learns, then uses, the context:

```
if (routingKey === 'order.created') {
  remember(orderContext, evt.orderId, { userId: evt.userId, items: evt.items ?? [] });
}
// ... later, for payment.failed on the same order:
const ctx = orderContext.get(orderId);
const to = ctx?.userId ? `user:${ctx.userId}` : 'user:unknown';
```

Run the FAILED demo (`LAPTOP-001 x1`) and check `curl.exe -s http://localhost:3006/notifications`: the *Payment failed* entry is attributed to your real user id, even though `payment.failed` itself never carried it — the state traveled in `order.created` and was joined locally.

10 senior interview questions

1. **A downstream consumer needs data the triggering event doesn't carry.

Enumerate your options and their costs.** (Call back to the owner — synchronous coupling + read-after-write races; shared DB — breaks ownership, schema coupling; carry state in events — decoupled, but payload growth + eventual consistency of the consumer's copy.)

2. **Thin vs fat events — when is each right?** (Thin when consumers are few/

internal and freshness matters more than autonomy; fat when consumer autonomy, audit, or fan-out breadth matters. Middle path: carry the fields downstream demonstrably needs.)

3. **How do you evolve an event schema without breaking consumers?**

(Additive-only changes; consumers ignore unknown fields; never repurpose a field; breaking changes get a new event name/version and a migration window where both are published.)

4. **Is the notification service's orderId->user map a source of truth?**

(No — a projection/cache derived from the stream. The order DB remains the truth; the map is rebuildable and safe to lose in proportion to its use.)

5. **What are the failure modes of consumer-local state, and how do you

bound them?** (Staleness — a later user-email change isn't reflected; loss on restart — attribute-unknown fallbacks; unbounded growth — cap + eviction. State the fallback behavior explicitly.)

6. **Why did inventory get a durable processed-events table while

notification got an in-memory set? Defend the asymmetry.** (Idempotency strength should match the cost of a duplicate: double-reserved stock is a data corruption; a duplicate email is an annoyance. Durable dedupe costs a DB round-trip per event — pay it where duplicates are expensive.)

7. **A duplicate `payment.failed` arrives after a restart wiped the dedupe

set. Walk through what happens.** (Set lookup misses -> notification sent again -> one repeated email. Order state is untouched — the saga's own idempotency lives in the orchestrator/DB, not here. Acceptable by stated design.)

8. **When does consumer-local projection graduate into CQRS?** (When the

projection needs durability, its own query API, rebuild/replay tooling, or serves user-facing reads. Same concept, industrialized: events -> materialized read model in its own store.)

9. **How would you replay/rebuild a consumer's state in this RabbitMQ-based

system, given queues don't retain acked messages?** (You can't from the broker — you'd re-drain from a durable source: the outbox table, an event store, or switch the backbone to a log like Kafka. Knowing the

broker's retention model drives this answer.)

10. **Carrying `userId` in `order.created` duplicates data the order DB

owns. Is that a smell?** (No — events are immutable facts, snapshots at a point in time, not live references. Duplication into events is how autonomy is bought; the smell would be consumers *writing* that data back somewhere authoritative.)